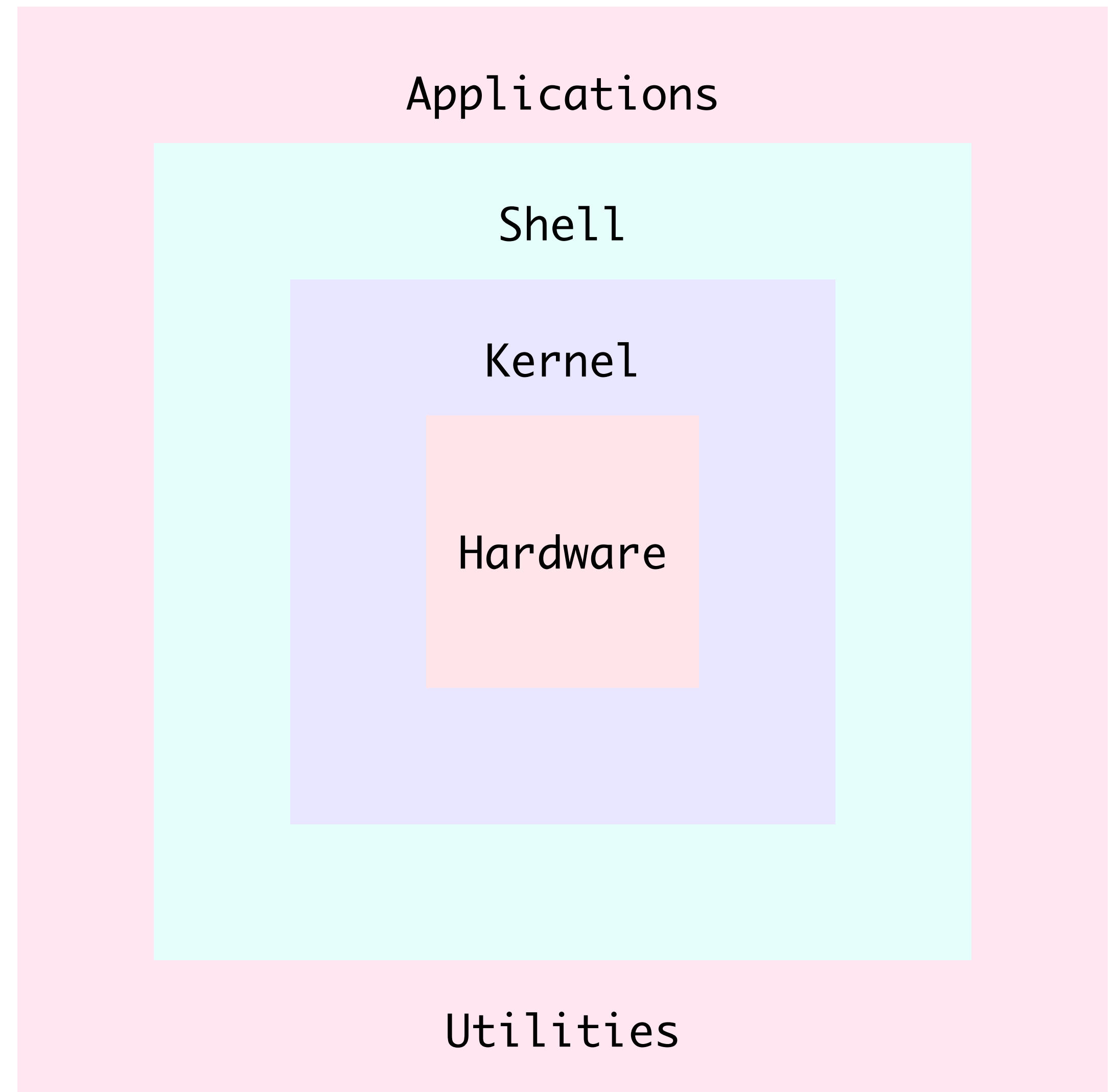


Linux

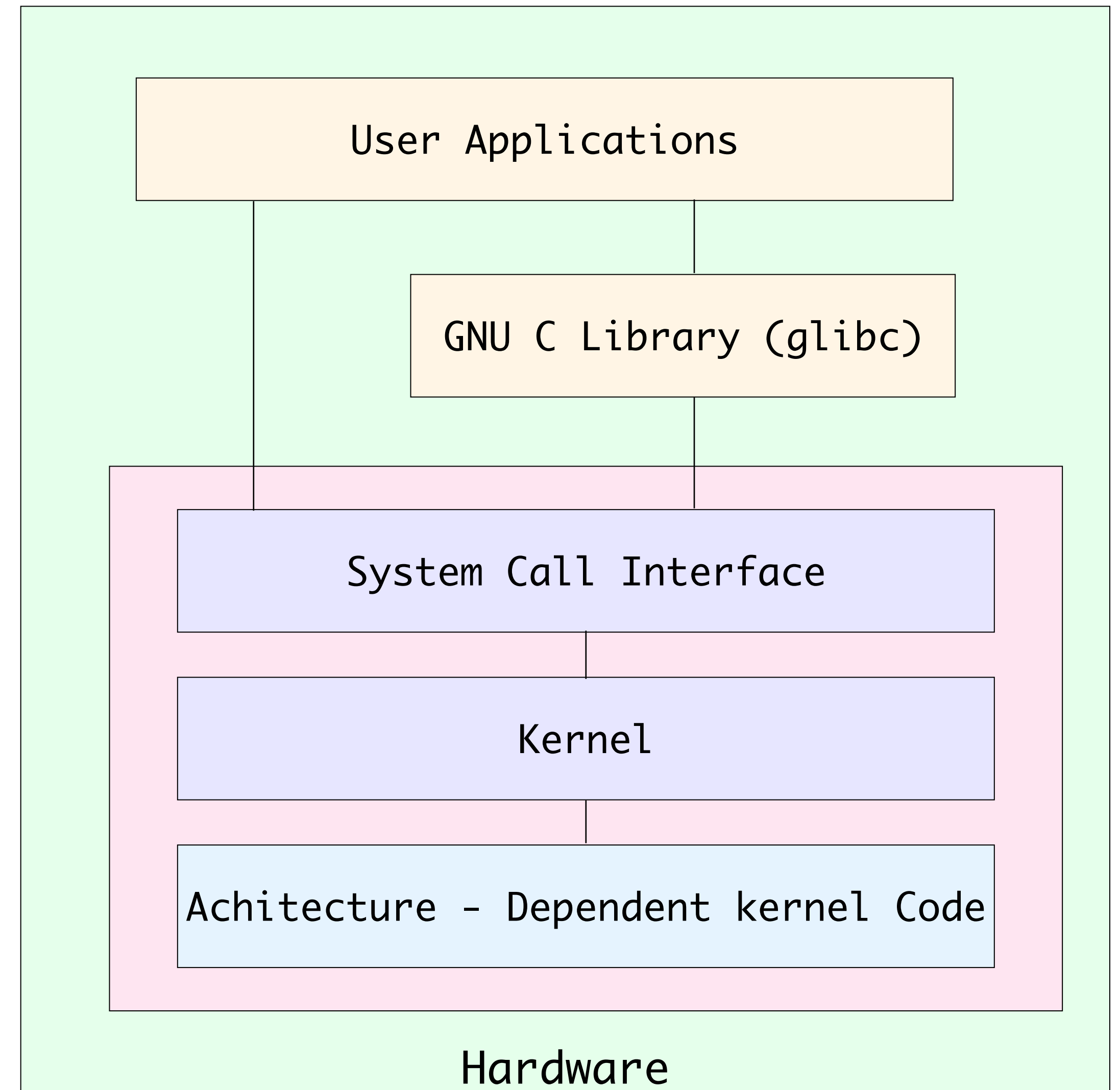
Linux Kernel

- What is the Linux Kernel?

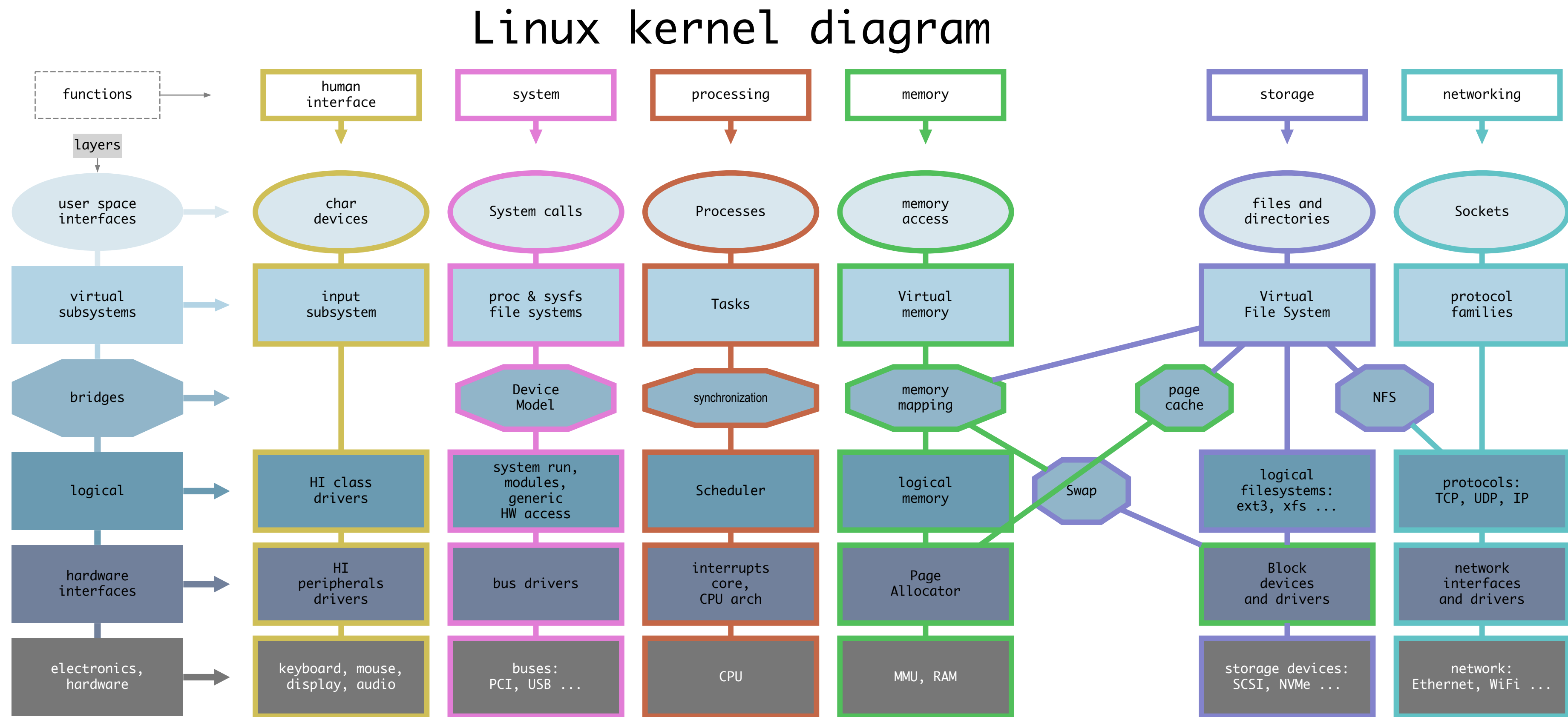


Linux Kernel

- User space vs. Kernel space
- Kernel's role in system administration



Kernel Architecture

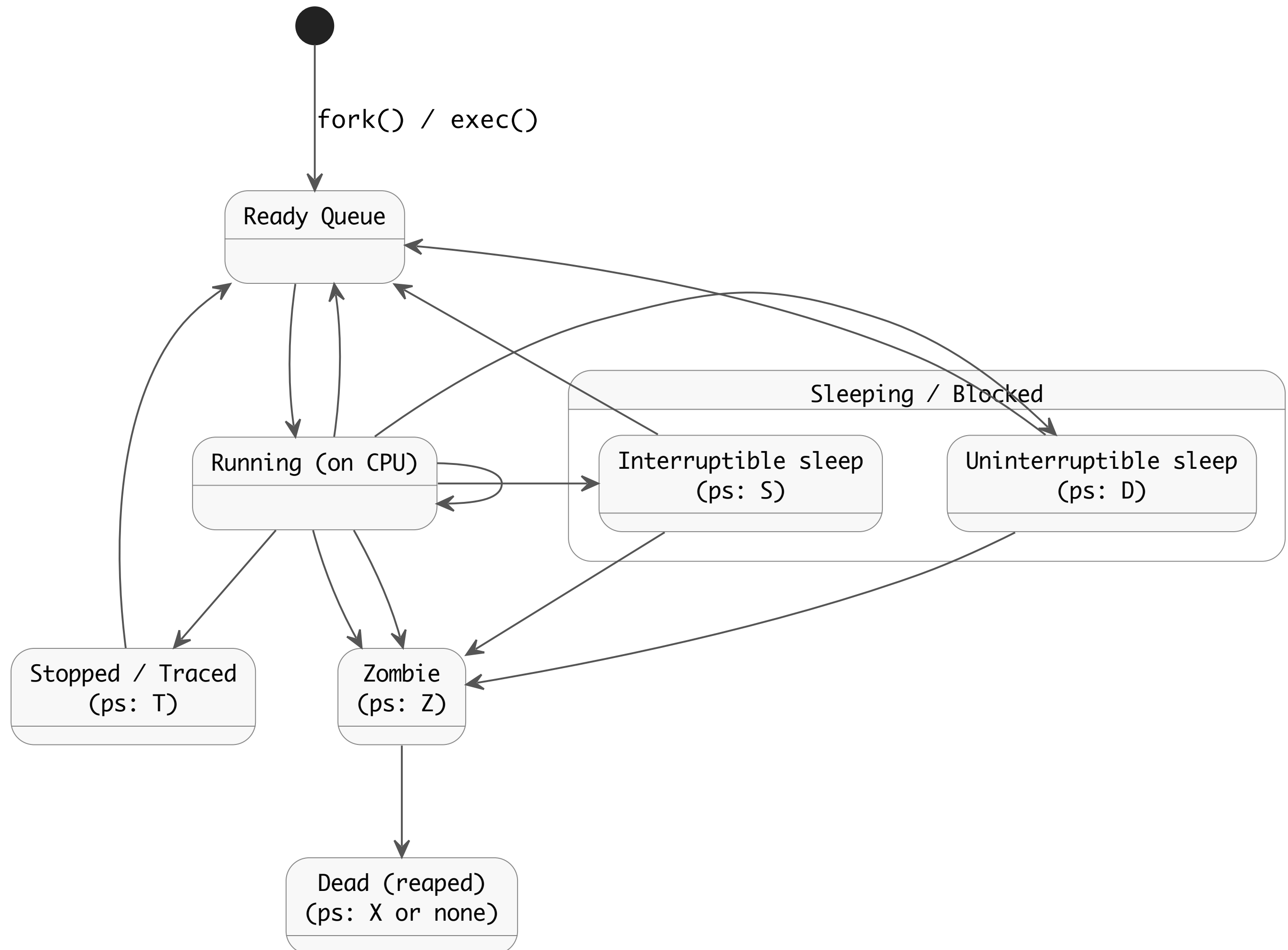


© 2007-2022 Costa Shulyupin <http://www.MakeLinux.net/kernel/diagram>

https://graphviz.org/Gallery/directed/Linux_kernel_diagram.svg

Process Management

Linux Process States (high-level view)



- Kernel
- User mode

Process Management

- Process Scheduler Classes
 - Real-time Process (Priority >0)
 - SCHED_FIFO
 - SCHED_RR
 - Normal Process (Priority 0)
 - SCHED_NORMAL / SCHED_OTHER
 - SCHED_BATCH
 - SCHED_IDLE

Process Management

- Normal Process Scheduler
 - CFS - Completely Fair Scheduler
 - EEVDF - Earliest Eligible Virtual Deadline First

User Level Process Management

- Inspect and list

- ps
- pstree
- top
- htop

- Find processes

- pgrep
- pidof
- ps aux

- Control / terminate

- kill -TERM <pid>
- kill -KILL <pid>
- killall <name>
- pgrep
- pkill

User Level Process Management

- Priorities & scheduling

- nice

- renice

- chrt

- ionice

- CPU affinity

- taskset

- Background & job control

- cmd &

- jobs

- fg %1

- bg %1

- disown -h %1

- nohup <cmd> &

User Level Process Management

- Resource limits
 - `ulimit -a`
 - `ulimit -n 4096`
 - `prlimit --pid <PID> --nofile=4096`
 - `prlimit --nproc=1024 --<cmd>`
- What a process is doing
 - `lsof -p <PID>`
 - `lsof -i :5432`
 - `pmap -x <PID>`
 - `strace -p <PID>`
 - `time <cmd>`

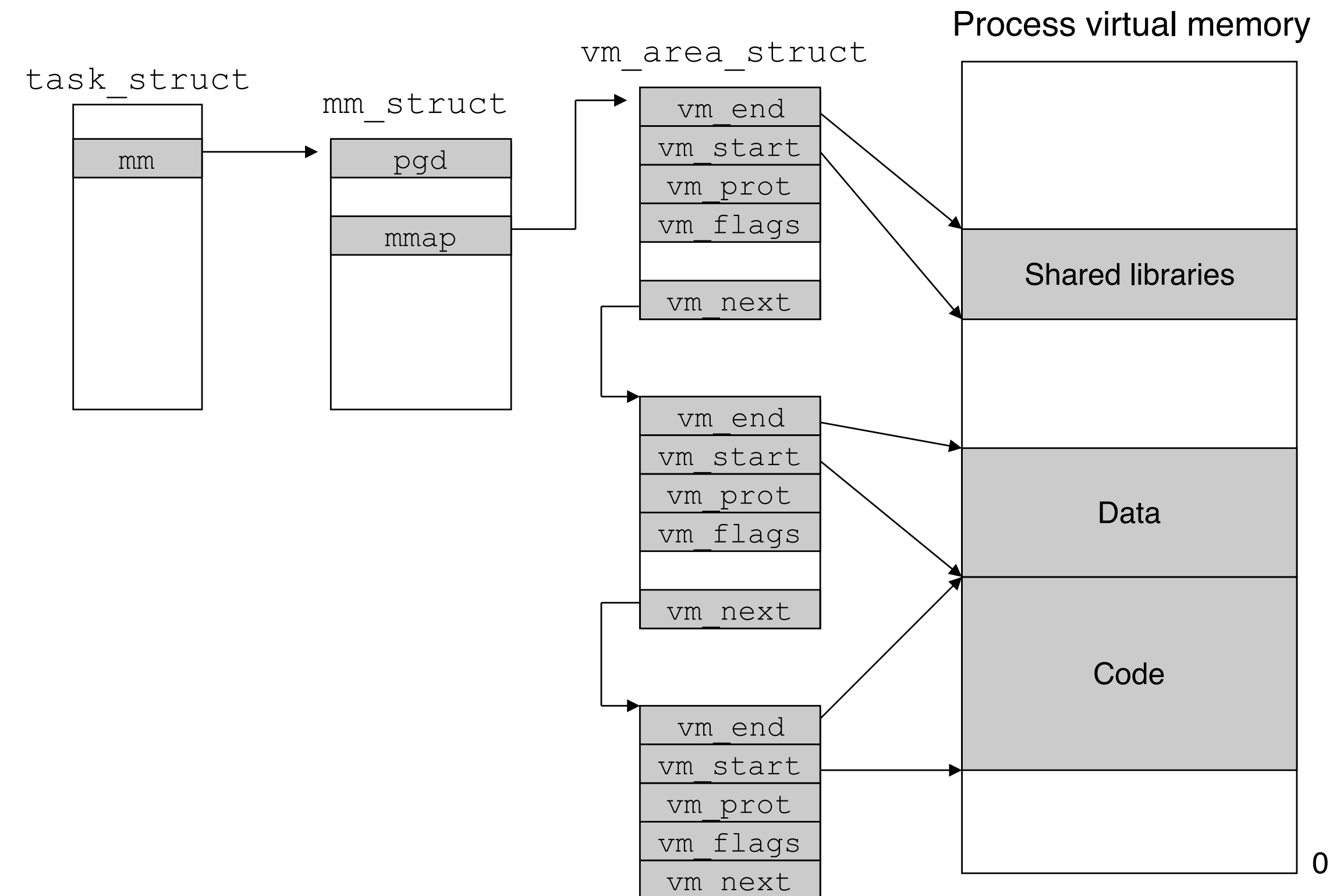
User Level Process Management

- Relationships & metadata
 - `ps -o pid,ppid,cmd -p <PID>`
 - `cat /proc/<PID>/status`
 - `cat /proc/<PID>/limits`
 - `cat /proc/<PID>/sched`
- `systemd user-session helpers`
- `systemctl --user status`
- `systemd-run --user <cmd>`
- `loginctl user-status $USER`

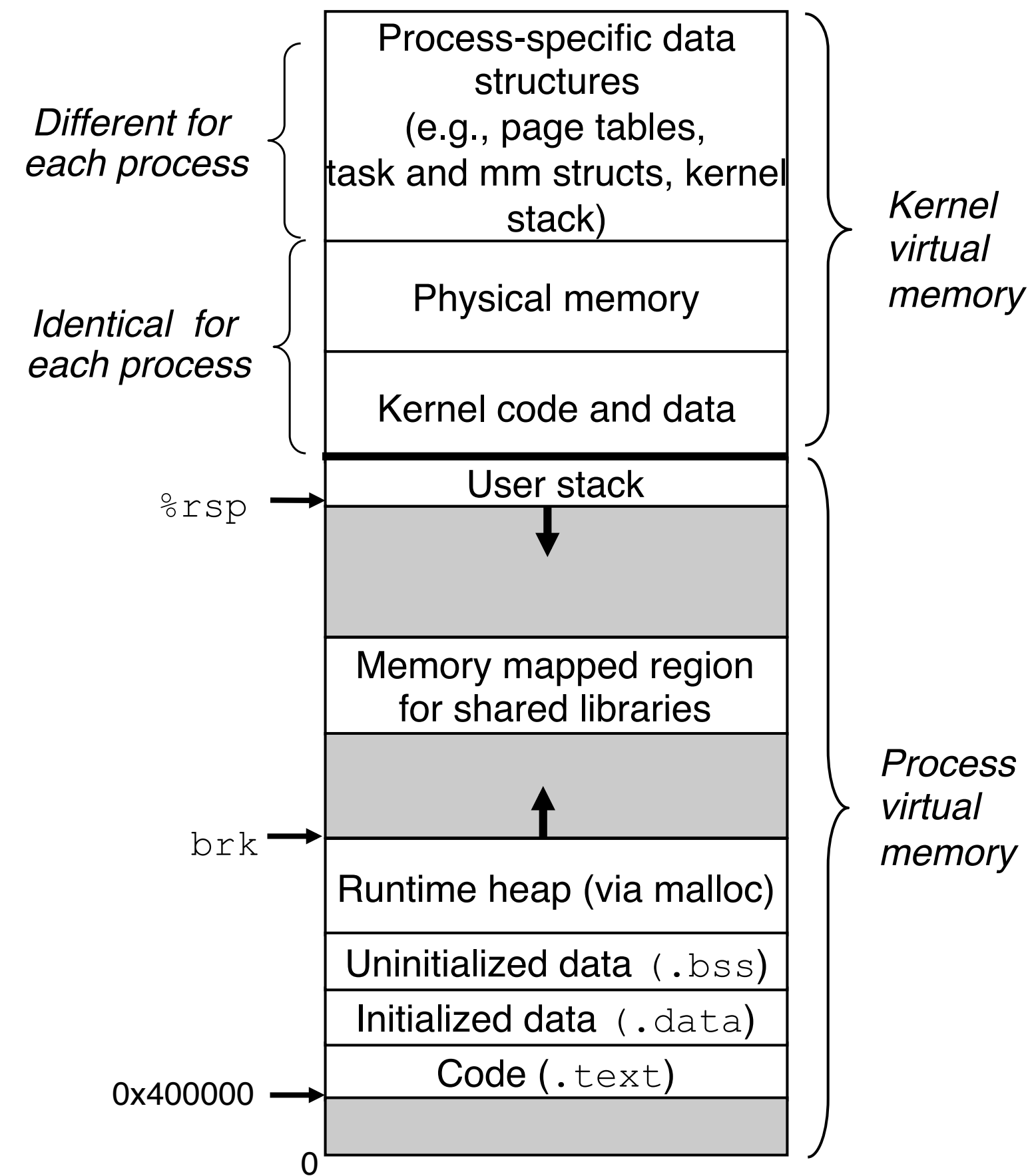
Memory Management

- Virtual memory, paging & page tables
- Swapping & page replacement
- Kernel memory allocators (slab, buddy)

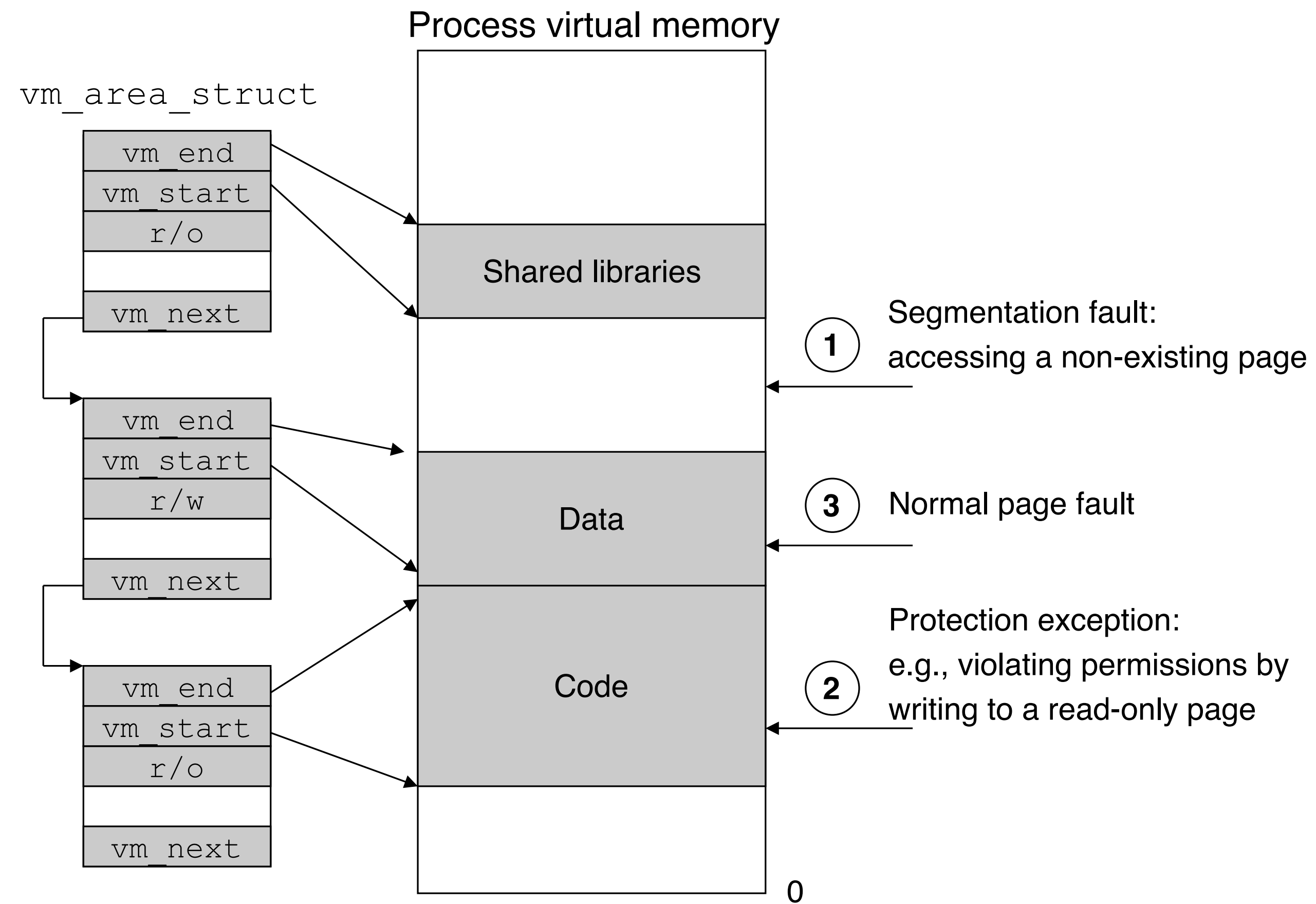
Memory Management: Process virtual memory



Memory Management: Process virtual memory

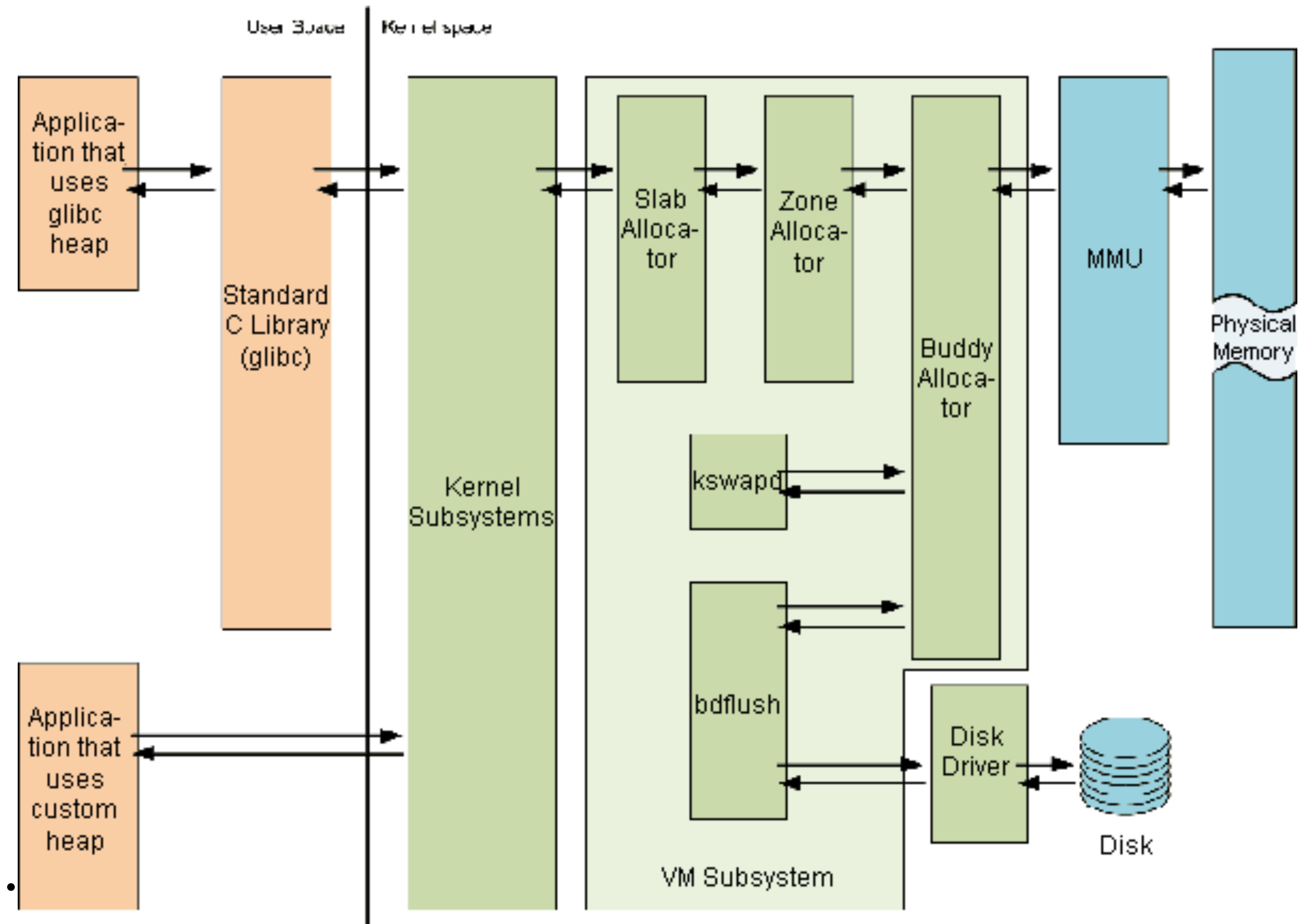


Memory Management: Page fault

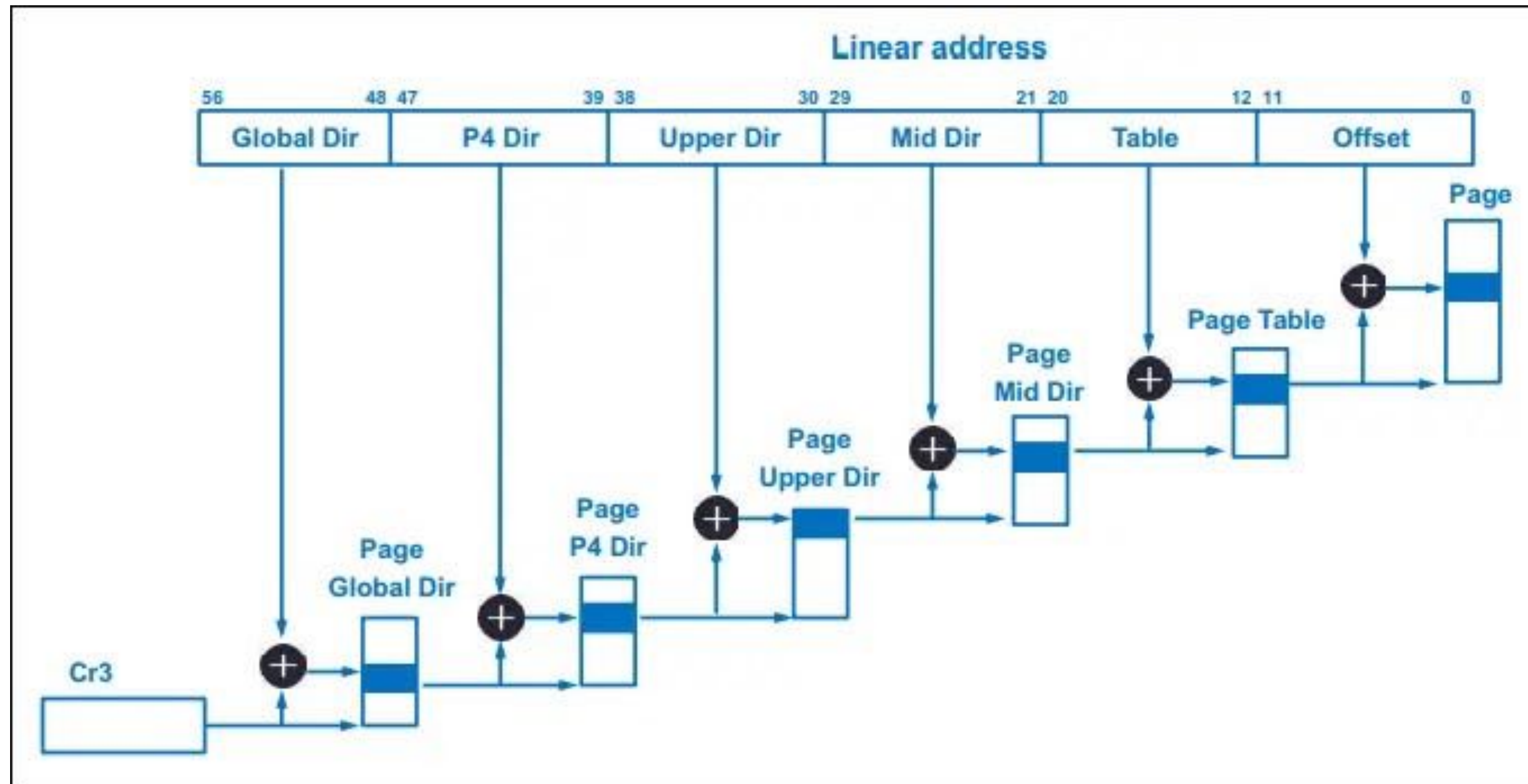


Memory Management: Virtual vs Physical

- Linux uses multi-level page tables
 - PGD - Page Global Directory
 - P4D - Page Level 4 Directory
 - PUD - Page Upper Directory
 - PMD - Page Middle Directory
 - PTE - Page Table Entry
- Address translation handled by MMU.
- TLB caches improve performance.



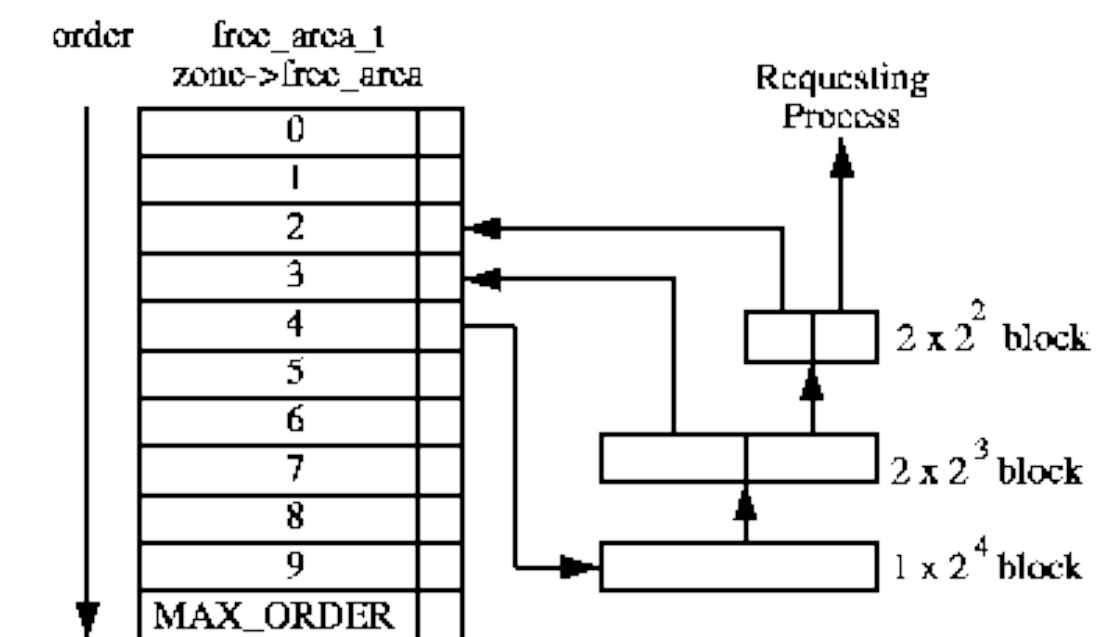
Memory Management: Virtual to Physical memory translation



Memory Management: Buddy System

- Page frames
 - Physical memory is divided into fixed-size blocks called page frames.
 - Each frame is tracked by a struct page in the kernel.
 - Common frame size: 4 KB on most architectures.

Node Zones
Zone DMA
Zone DMA
Zone Normal
Zone HighMemory



Memory Management: User level commands

- free
- vmstat
- top
- htop
- /proc/meminfo
- /proc/[pid]/maps
- pmap
- slabtop

I/O SubSystem

- Core Interface for Hardware Communication: bridge between applications and physical devices, handling data transmission across hardware interfaces.
- Handles multiple device Types: character and block devices, network devices, and special files using a unified interface
- Crucial for Performance and Reliability: Efficient I/O subsystems minimize latency, maximize throughput and ensure data integrity during RW operations
- Extensible Through Kernel Modules: Device-specific logic is often implemented via loadable kernel modules
- Supports Multiple Filesystems and Devices. Abstracts hardware diversity to allow consistent interaction with filesystems like ext4, XFS, and devices like SSDs, HDDs, NVMEs

I/O Performance

- I/O as a Critical Bottleneck: In many systems, I/O speed—not CPU performance—limits overall throughput, especially in database and large-scale analytics workloads.
- Impact on Application Responsiveness: Slow disk or network I/O can introduce significant latency, degrading the user experience and increasing response times.
- Key to Scalability in High-Load Environments: Optimized I/O paths enable systems to scale horizontally without performance collapse under concurrency.
- Influences Resource Utilization: Inefficient I/O operations increase CPU usage and energy consumption, reducing system efficiency and lifespan.
- Benchmarking Reveals System Health: Tools like fio, iostat, and dstat help identify I/O constraints that might otherwise go unnoticed during development.

Block vs. Character Devices

- Key Distinctions in Linux Device Management
 - Block Devices: Random Access with Buffering: Used for storage media like hard drives and SSDs, block devices support random access and benefit from caching and buffering via the kernel.
 - Character Devices: Sequential Stream Access: Character devices such as keyboards, mice, and serial ports transmit data byte-by-byte with no buffering, enabling real-time interaction.
 - Major and Minor Numbers: Each device is identified by a pair of numbers; the major number links to the driver, while the minor distinguishes between devices.

Block vs. Character Devices

- Access via /dev Interface: Devices appear as special files under /dev, allowing consistent user-space access to hardware resources.
- Examples and Use Cases:
 - Block: /dev/sda, /dev/nvme0n1;
 - Character: /dev/tty, /dev/null.
 - Each suits different data access patterns.

Device Drivers in Linux

Role, Types, and Integration in the Kernel

- Bridge Between Hardware and Kernel: Device drivers translate hardware signals into kernel-readable data, enabling communication and control without direct hardware manipulation.
- Types of Linux Device Drivers: Includes character, block, and network drivers, each tailored to specific I/O patterns and data structures.
- Modular and Loadable Kernel Modules: Drivers can be compiled as modules (.ko) and dynamically loaded using insmod or modprobe without rebooting.
- Unified Driver Model: The Linux kernel provides a structured API for driver development, standardizing registration, I/O ops, and cleanup.
- Device Trees and Udev Rules: Modern kernels use device trees and udev for dynamic hardware detection and configuration.

I/O Scheduling and Queues

Mechanisms for Efficient Disk Access

- Role of I/O Scheduler: Schedules pending I/O operations to optimize throughput and latency by minimizing disk seek times and balancing load.
- Common Linux Schedulers: Includes cfq, deadline, noop, and bfq, each designed for different workloads and hardware (e.g., SSD vs HDD).
- Request Queues: Each block device maintains a request queue where I/O operations are sorted, merged, or delayed based on scheduler policies.
- Elevator Algorithm and Merging: I/O requests are often reordered to follow disk arm movement efficiently; adjacent requests may be merged.
- Dynamic Tuning via sysfs: Schedulers and queue parameters can be adjusted at runtime under `/sys/block/queue/` to match workload needs.

Linux Schedulers

- NOOP
 - Simple FIFO queue
 - Minimal CPU overhead
 - Ideal for SSDs and virtualized environments
- Deadline
 - Two queues: read/write with deadlines
 - Prevents starvation
 - Good for latency-sensitive workloads
- CFQ (Completely Fair Queuing)
 - Per-process queues
 - Good for desktop or mixed workloads
- BFQ (Budget Fair Queuing)
 - Bandwidth-based fairness
 - Great for multimedia and real-time apps
- Kyber & MQ-Deadline
 - Designed for multi-queue block layer
 - Scalable for NVMe devices

Scheduler Comparison

Scheduler	Fairness	Latency	Throughput	Best For
NOOP	Low	Low	High	SSDs, VMs
Deadline	Medium	Low	High	Databases
CFQ	High	Medium	Medium	Desktops
BFQ	Very High	Medium	Medium	Multimedia
Kyber	Medium	Very Low	High	NVMe, servers

The VFS Layer in Linux

Abstraction and Uniformity Across Filesystems

- What is the Virtual File System (VFS)?: A kernel-level abstraction layer that provides a unified interface for all filesystems, enabling consistency across ext4, XFS, NFS, and more.
- Decouples Applications from Filesystem Internals: Applications interact with VFS APIs, not the underlying filesystems directly, improving portability and reducing complexity.
- Inodes, Dentries, and Superblocks: VFS manages metadata structures like inodes (file info), dentries (directory entries), and superblocks (filesystem state).
- Mounting Mechanism and Namespaces: VFS supports mounting multiple filesystems into a single hierarchical namespace and handles isolation via mount namespaces.
- Pluggable Filesystem Modules: Supports dynamically loadable filesystem drivers that register with the VFS, enabling flexibility and modularity.

Page Cache and Buffer Cache in Linux

Memory Optimization Mechanisms for I/O

- Page Cache: File Data Caching: Stores file contents read from disk in memory, allowing faster subsequent reads and write-back operations.
- Buffer Cache: Metadata and Block Buffers: Holds disk block data and filesystem metadata, enhancing performance for block-based storage operations.
- Unified Caching Since 2.4 Kernel: Modern Linux kernels treat page and buffer cache as part of the same memory pool, dynamically allocating space based on usage.
- Minimizing Disk Access: By caching frequently accessed data, the system reduces disk I/O frequency, improving speed and decreasing wear on SSDs.
- Tools to Monitor Cache Usage: Utilities like `free`, `vmstat`, and `/proc/meminfo` provide insights into cache utilization and reclaim behavior.

Direct I/O vs Buffered I/O

Choosing the Right I/O Path for Performance

- Buffered I/O: Kernel Caching Layer: Default Linux I/O model where data is cached in page cache before being written to disk or read by applications.
- Direct I/O: Bypasses Page Cache: Provides applications with direct disk access, avoiding kernel buffering—essential for databases and low-latency workloads.
- Trade-offs: Speed vs Safety: Buffered I/O offers better average performance through caching, while direct I/O ensures consistent latency and reduced double-buffering.
- File System Support Required: Not all filesystems or kernel configurations support direct I/O; ext4, XFS, and btrfs do with specific mount or open flags.
- When to Use Direct I/O: Ideal for high-throughput systems like databases (e.g., Oracle, PostgreSQL) where application-level caching is already used.

Asynchronous I/O (AIO) and io_uring

Revolutionizing High-Performance I/O in Linux

- Traditional AIO Limitations: Linux's original AIO implementation lacked true async behavior for file I/O, leading to complexity and poor performance.
- Introduction of io_uring: Introduced in Linux 5.1, io_uring provides a fast, flexible API for truly asynchronous I/O with submission and completion queues.
- Efficient System Call Interface: Reduces syscall overhead by batching I/O operations, enabling thousands of concurrent ops with minimal CPU usage.
- Supports Advanced Features: Includes multishot ops, buffer selection, fixed file descriptors, and SQPOLL for user-space polling and zero-copy.
- Adoption by Performance-Critical Apps: Increasingly used in modern databases, file servers, and NVMe stack for scalable, low-latency I/O.

DMA and Zero-Copy Techniques

Accelerating I/O Through Direct Memory Access

- What is DMA (Direct Memory Access)?: A technique where devices transfer data directly to/from RAM without CPU intervention, reducing overhead.
- Benefits of DMA: Frees up CPU cycles, lowers latency, and enhances throughput by removing the CPU from the data path.
- Zero-Copy I/O in Linux: Eliminates redundant memory copies between user space and kernel space using `mmap`, `sendfile()`, `splice()`, and `io_uring`.
- Network and Storage Optimization: Widely used in high-speed networking (e.g., RDMA) and storage I/O to reduce system resource usage and increase efficiency.
- Kernel Support and Drivers: DMA and zero-copy support must be implemented in hardware and drivers to realize their performance gains.

Filesystem Impact on I/O Performance

- ext4: General Purpose Workhorse: Offers excellent performance for general workloads with mature journaling, low overhead, and wide compatibility.
- XFS: High-Performance for Large Files: Optimized for parallelism and scalability, XFS excels in environments with large files and high-throughput needs.
- btrfs: Advanced Features and Flexibility: Supports snapshots, compression, and checksumming; ideal for systems needing advanced data integrity and management.
- Trade-offs Between Filesystems: ext4 is stable and fast; XFS scales well for enterprise loads; btrfs offers features at the cost of higher complexity and tuning needs.
- Tuning and Mount Options Matter: Performance can vary significantly based on options like journaling mode, I/O scheduler, and allocation policies.

Monitoring and Profiling I/O in Linux

Tools to Understand and Optimize I/O Behavior

- `iostat`: Disk Utilization Metrics: Displays CPU and I/O statistics per device, helping identify slow disks, busy queues, and latency trends.
- `blktrace`: Block Layer Tracing: Provides low-level block I/O traces for analyzing how I/O requests are processed in the kernel stack.
- `perf`: Comprehensive Performance Analysis: Monitors kernel and user-space events, including I/O latency, syscalls, and CPU stalls during disk access.
- `pidstat`, `vmstat`, `sar`: Track per-process I/O usage, memory pressure, and historical system stats for holistic system diagnostics.
- Visualizing I/O Behavior: Tools like `iostat`, `dstat`, and grafana dashboards help visualize trends and identify bottlenecks interactively.

Networking Subsystem

- Linux kernel provides a complete, modular networking stack
- Supports standard and high-performance interconnects
- Used across servers, routers, and supercomputers
- Focus:
 - Core networking subsystems
 - HPC fabric support (InfiniBand, Omni-Path, Tofu, ...)

Kernel Networking Architecture

- Layers:

1. Network device drivers (L2)
2. Protocol stack (L3/L4)
3. Socket interface (user space API)

- Managed under `/net/` in kernel source

- Modular and hardware-independent

Major Networking Subsystems

Subsystem	Role
Netdevice	Interface abstraction for all NICs
TCP/IP Stack	IPv4/IPv6, UDP, TCP, SCTP
Netfilter	Packet filtering and NAT
Routing	Forwarding and policy routing
Traffic Control	QoS and shaping
Virtual Networking	Bridge, VLAN, Bonding, Namespaces
RDMA Core	High-performance fabric support

Network Device Layer

- `struct net_device` abstraction
- Drivers register transmit/receive callbacks
- Supports:
 - Ethernet (Intel, Broadcom, etc.)
 - Wireless (`cfg80211/mac80211`)
 - Virtual interfaces (veth, tun/tap)
 - HPC interconnects (InfiniBand, Omni-Path, Tofu)

Socket Layer

- Kernel-level implementation of BSD sockets
- Entry point for user-space networking (socket(), send(), recv())
- Types:
 - INET: TCP/UDP/IP
 - PACKET: Raw Ethernet access
 - NETLINK: Kernel-user communication
 - RDMA: Remote direct memory access

Protocol Stack Implementation

- Layer 3–4 protocols: IPv4, IPv6, TCP, UDP, SCTP, DCCP
 - Implements congestion control, routing, fragmentation, reassembly
- Tunneling protocols: GRE, IPIP, VXLAN, GENEVE
 - Integrated IPsec and encryption support

Netfilter Framework

- Kernel framework for packet filtering and NAT
- Hooks:
 - PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING
- Implements:
 - Stateful firewalls
 - NAT, masquerading
 - Connection tracking
- Used by iptables, nftables, and conntrack

Traffic Control (QoS)

- Framework for shaping and prioritizing packets
- Key components:
 - Queue disciplines (HTB, fq_codel, TBF)
 - Classifiers and filters
- User-space tool: tc
- Kernel modules under /net/sched/

Virtual Networking

- Kernel modules for:
 - Bridging: Software Layer 2 switch
 - Bonding: Link aggregation
 - VLANs: IEEE 802.1Q tagging
 - Namespaces: Container isolation
 - VXLAN / Geneve: Overlay networks for virtualization

RDMA and High-Performance Networking

- Remote Direct Memory Access (RDMA) framework in kernel
- Allows zero-copy data transfer bypassing kernel TCP/IP stack
- Implemented via:
 - `drivers/infiniband/`
 - `drivers/net/ethernet/rdma/`
- User APIs: RDMA CM, libibverbs, OFED stack

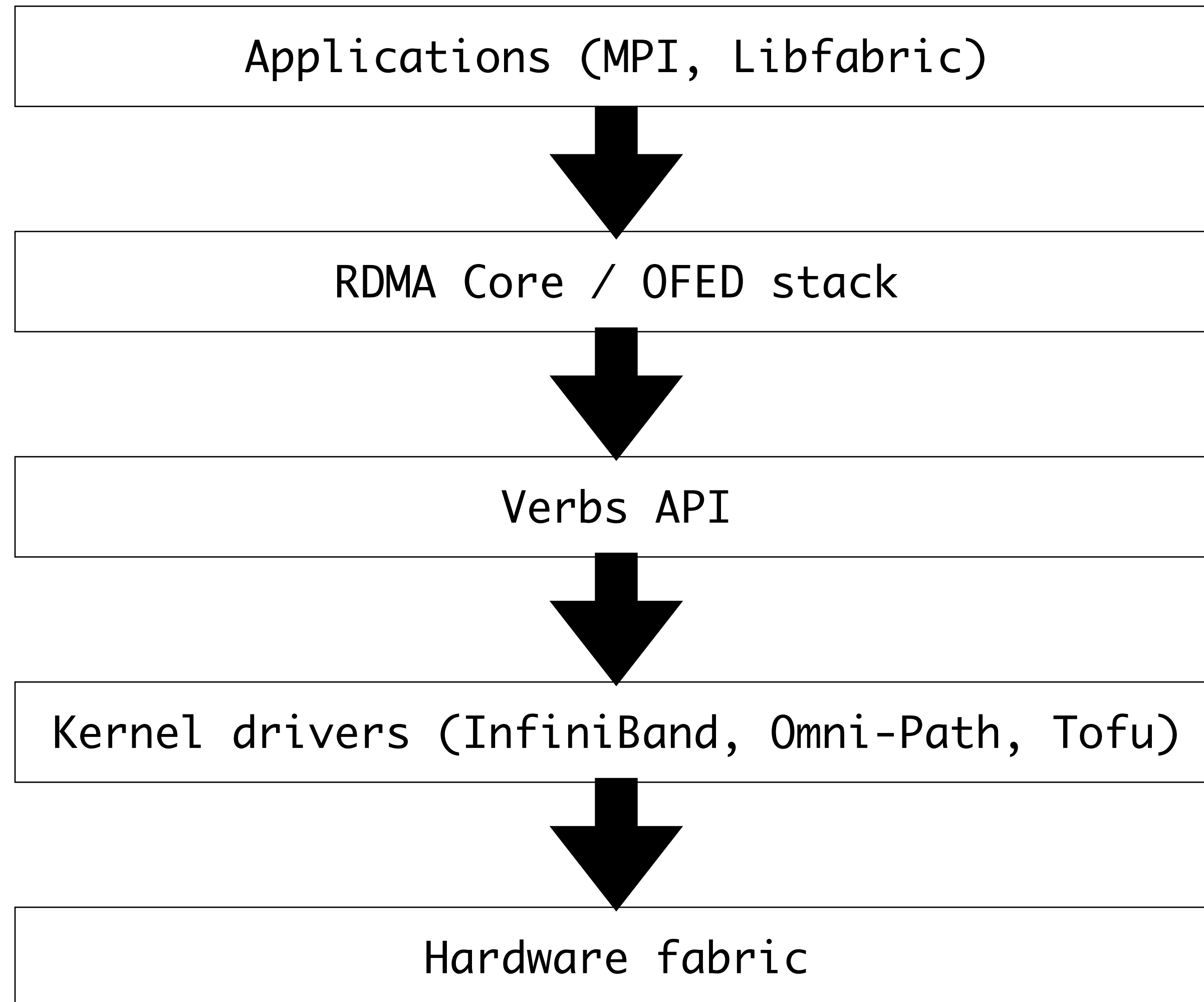
InfiniBand Support

- Integrated via RDMA core subsystem
- Kernel modules:
 - `ib_core`, `ib_ipoib` (IP over IB)
- Features:
 - Low latency (microseconds)
 - High bandwidth (>200 Gbps)
 - Reliable connection and datagram modes
- Supports verbs API and SR-IOV for virtualization

Intel Omni-Path Architecture (OPA)

- Supported in Linux via hfi1 driver and RDMA layer
- Similar to InfiniBand but with Intel extensions
- Key kernel components:
 - `drivers/infiniband/hw/hfi1/`
 - `opa_vnic` for Ethernet emulation
- Integrated with OFED stack and verbs API
- Used in HPC and data center fabrics

RDMA & HPC Stack Overview



Performance Enhancements

- NAPI: Interrupt mitigation and polling
- RPS/RFS: Multi-core packet distribution
- Multi-Queue NIC support
- XDP (Express Data Path) for high-performance packet processing
- eBPF for programmable network hooks

Kernel Configuration Options

- `CONFIG_INFINIBAND`, `CONFIG_RDMA_CM`, `CONFIG_NETFILTER`
- `CONFIG_XDP_SOCKETS`, `CONFIG_VLAN_8021Q`, etc.
- Loadable modules for flexible kernel builds

User level tools

- Tools:
 - ethtool, ss, ip, /proc/net/*
 - ibstat, ibv_devinfo, perf, bpftrace
 - rdma dev show (RDMA configuration)
- Kernel tracing with tracepoints and perf record